Task 1.1

Proof: ($\Rightarrow$) $y = f(x) + \varepsilon$, $\varepsilon \sim N(0, \Sigma)$

We have $\varepsilon = y - f(x)$.

If x is given, $y = \varepsilon + f(x) \sim N(f(x), \Sigma)$.

(By Transformation laws: $Y = BX + c$, $X \sim N(\mu, \Sigma)$, then $Y \sim N(c + B\mu, B\Sigma B^T)$

($\Leftarrow$) $y = f(x) + \varepsilon$, $y|x \sim N(f(x), \Sigma)$.

We can show it similarly.

Task 1.2. $\quad \in \mathbb{R}^{n \times n}$

$y|X = X\beta + \epsilon \sim N(X\beta, \sigma^2 I_n)$ from the result of 1.1. Thus:

$$p(y|X) = \frac{1}{\sqrt{(2\pi)^n/\sigma^2}} \exp\left(-\frac{\sigma^2}{2}(y - X\beta)^T(y - X\beta)\right)$$

$$\log p(y|X, \beta, \sigma^2) = -\frac{\sigma^2}{2}(y - X\beta)^T(y - X\beta) - \frac{n}{2}\log(2\pi) + \log\sigma. \quad (\#)$$

To minimize (#) is equivalent to maximize $\|y - X\beta\|_2^2$ as $-\sigma^2/2$ is a negative constant factor and $-n/2 \log(2\pi) + \log\sigma$ is constant.

Task 1.3.

$$\hat{\beta}_{MLE} = \operatorname{argmax}_\beta \|y - X\beta\|_2^2$$

$$\frac{\partial}{\partial\beta}\|y - X\beta\|_2^2 = -2X^Ty + 2X^TX\beta = 0 \Rightarrow \hat{\beta}_{MLE} = (X^TX)^{-1}X^Ty.$$

This estimator is unique since X has full column rank. Hence X^TX is invertible.

$X^TX\beta = X^Ty$ (normal equation)

To derive the covariance, substitute y by $X\beta + \varepsilon$.

$$\hat{\beta}_{MLE} = (X^TX)^{-1}X^T(X\beta + \varepsilon)$$

$$= \beta + (X^TX)^{-1}X^T\varepsilon$$

$$Cov(\hat{\beta}_{MLE}) = Cov(\beta + (X^TX)^{-1}X^T\varepsilon)$$

$$= Cov((X^TX)^{-1}X^T\varepsilon)$$

$$= (X^TX)^{-1}X^T \, Cov(\varepsilon) \, X \, (X^TX)^{-1}$$

$$= (X^TX)^{-1}X^TX(X^TX)^{-1} \, Cov(\varepsilon)$$

$$= \sigma^2(X^TX)^{-2}. \qquad \sigma^2 I$$

Ex2

Task1

$\mu_0 = 0 \quad \Sigma_0 = r^2 I_p \quad \mu_{MLE} = \hat{\beta}_{MLE} \quad \Sigma = \sigma^2 (X^T X)^{-1}$

$\Rightarrow \Sigma_{post} = \left(\frac{1}{r^2} I_p + \frac{1}{\sigma^2} X^T X\right)^{-1}$

$\mu_{post} = \Sigma_{post} \left(\frac{1}{\sigma^2} X^T y\right)$

$p(\beta | y, X) = \mathcal{N}(\mu_{post}, \Sigma_{post})$

Task2

$p(y_* | x_*, X, y) = \int p(y_* | x_*, \beta)\, p(\beta | X, y)\, d\beta$

$p(y_* | x_*, \beta) = \mathcal{N}(x_*^T, \beta, \sigma^2)$

$p(\beta | X, y) = \mathcal{N}(\mu_{post}, \Sigma_{post})$

Integrating over the product of two Gaussians gives:

$p(y_* | x_*, X, y) = \mathcal{N}(m(x_*), v(x_*))$

$m(x_*) = \mathbb{E}[y_* | x_*] = x_*^T \mu_{post}$

$v(x_*) = \mathbb{E}_\beta[\mathrm{Var}[y_* | \beta]] + \mathrm{Var}_\beta[\mathbb{E}[y_* | \beta]] = \sigma^2 + x_*^T \Sigma_{post} x_*$

Task3

a) This ensures that the likelihood is still gaussian as well as the posterior over w

b) $\Sigma_{post} = \left(\frac{1}{r^2} I_D + \frac{1}{\sigma^2} \Phi^T \Phi\right)^{-1} \quad \mu_{post} = \frac{1}{\sigma^2} \Sigma_{post} \Phi^T y$

$m(x_*) = \mathbb{E}[y_* | x_*] = \phi(x_*)^T \mu_{post} = \frac{1}{\sigma^2} \phi(x_*)^T \Sigma_{post} \Phi^T y$

c) $A = \frac{1}{r^2} I_D + \frac{1}{\sigma^2} \Phi^T \Phi \qquad k(x, x') = \phi(x)^T \phi(x')$

$\Rightarrow m(x_*) = \phi(x_*)^T A^{-1} \Phi^T y \qquad K = \Phi \Phi^T$

$A^{-1} \Phi^T = \Phi^T \left(K + \frac{\sigma^2}{r^2} I_n\right)^{-1} \qquad k_* = \Phi\, \phi(x)_* \qquad (k_*)_i = k(x_i, x_*)$

$m(x_*) = k_*^T \left(K + \frac{\sigma^2}{r^2} I_n\right)^{-1} y \Rightarrow$ depends only on products of the form $\phi(x_i)^T \phi(x_j)$

d) $\Sigma_{post}\Phi^T = \Phi^T\left(K + \frac{\sigma^2}{r^2} I_n\right)^{-1}$

$m(x_*) = \phi(x_*)^T \Sigma_{post} \Phi^T y = \phi(x_*)^T \Phi^T \left(K + \frac{\sigma^2}{r^2} I_n\right)^{-1} y$

$k_* = [k(x_1, x_*), \dots, k(x_n, x_*)]^T$ $\quad k(x_i, x_*) = \phi(x_i)^T \phi(x_*)$

$\Rightarrow m(x_*) = k_*^T \left(K + \frac{\sigma^2}{r^2} I_n\right)^{-1} y$

e) $k(x, x')$ as a covariance:

$f(x) \sim GP(0, k(x, x'))$

$f(x_1) \dots f(x_n)$ are jointly Gaussian with covariance k

$D \to \infty$ (infinite-dimensional $\phi(x)$):

thanks to the kernel trick we avoid computing $\phi(x)$ directly

Ex2 Task 4.

σ^2 is the part correspond to aleatoric because it is the original data noises and therefore does not change because it is part of the data.

$\phi_A^T \Sigma_{post} \phi_A$ is the epistemic uncertainty part. This is because it is the posterior variance which is the uncertainty of weight.

The former does not shrink when $n \to \infty$ because it is part of data and can not be changed. The latter change because as $n \to \infty$, uncertainty decrease, thus it shrinks.

Machine Learning Essentials SS25 - Exercise Sheet 8

Instructions

- `TODO`**'s indicate where you need to complete the implementations.**
- **You may use external resources, but write your own solutions.**
- **Provide concise, but comprehensible comments to explain what your code does.**
- **Code that's unnecessarily extensive and/or not well commented will not be scored.**

In [6]:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns # For better aesthetics
from sklearn.linear_model import LinearRegression
from scipy.special import binom # Binomial coefficients for polynomial features

sns.set_theme(style="whitegrid")
```

Exercise 1

Task 4

In [7]:

```
# Generate Synthetic data
n = 60
beta0, beta1, sigma = 1.0, -0.8, 1.0 # Choose Some values for the true params
x = np.linspace(-3, 3, n) # Clamp imput x to some range

y = beta0 + x * beta1 + np.random.normal(0, sigma, n) # TODO: Implement data generating function
```

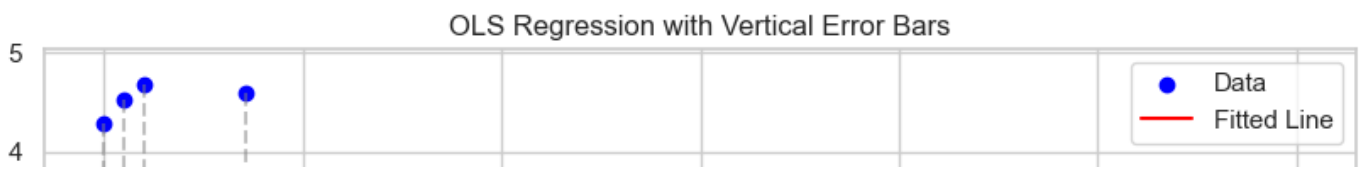
In [8]:

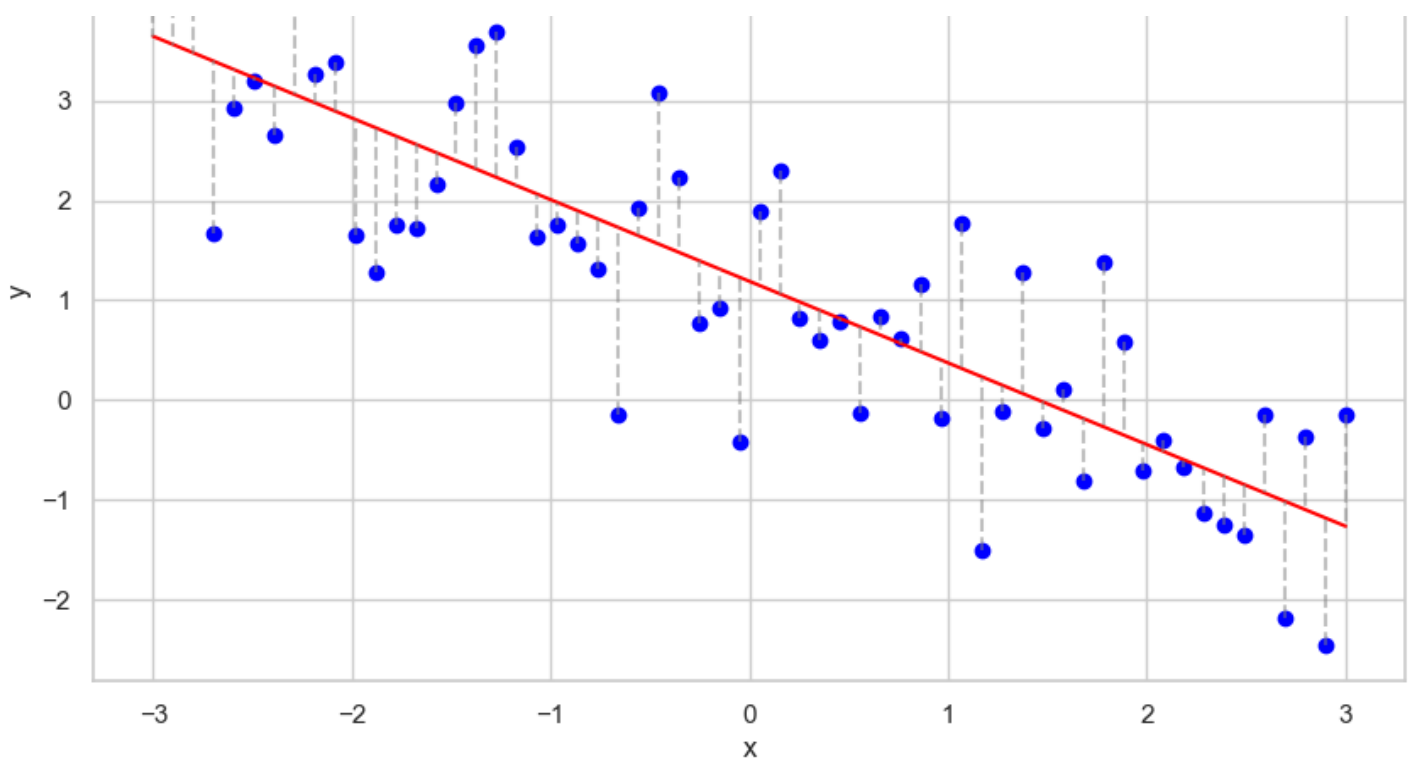
```
# TODO: Fit OLS model using sklearn
reg = LinearRegression().fit(x.reshape(-1, 1), y.reshape(-1, 1))
print(f"Score for regularization is: {reg.score(x.reshape(-1, 1), y.reshape(-1, 1))}, coefficients are: {reg.coef_}")
```

```
Score for regularization is: 0.7175404160967007, coefficients are: [[-0.8182881]]
```

In [9]:

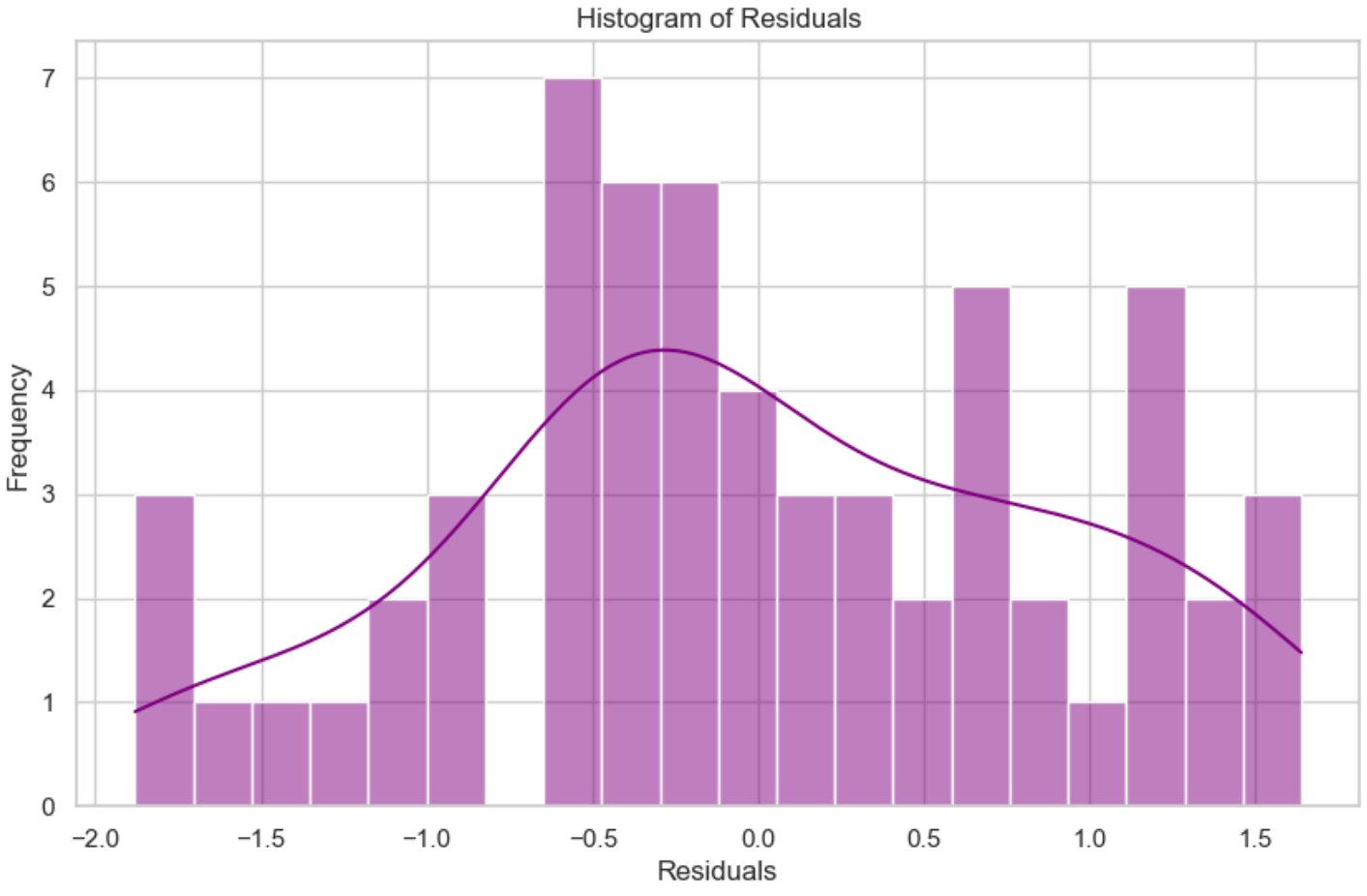
```
# TODO: Plot data, fitted line and vertical error (residual) bars
plt.figure(figsize=(10, 6))
plt.scatter(x, y, label='Data', color='blue')
plt.plot(x, reg.predict(x.reshape(-1, 1)), color='red', label='Fitted Line')
for i in range(n):
    plt.plot([x[i], x[i]], [y[i], reg.predict(x[i].reshape(-1, 1))[0, 0]], color='gray', linestyle='--', alpha=0.5)
plt.xlabel('x')
plt.ylabel('y')
plt.title('OLS Regression with Vertical Error Bars')
plt.legend()
plt.show()
```





In [10]:

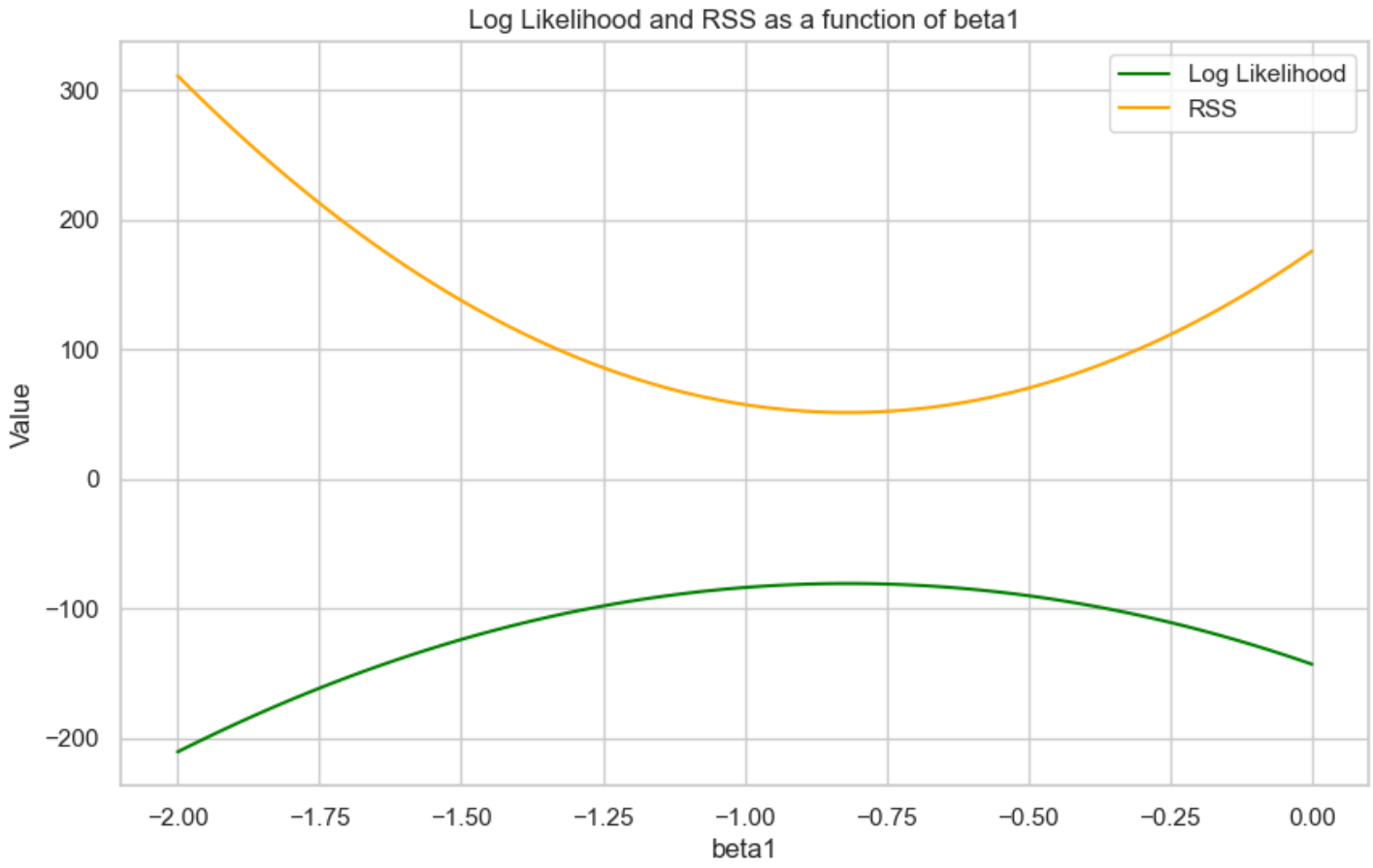
```
# TODO: Do a histogram of the residuals (e.g. sns.histplot)
residuals = y - reg.predict(x.reshape(-1, 1)).flatten()
plt.figure(figsize=(10, 6))
sns.histplot(residuals, bins=20, kde=True, color='purple')
plt.xlabel('Residuals')
plt.ylabel('Frequency')
plt.title('Histogram of Residuals')
plt.show()
```



In [11]:

```
# TODO: Plot the log likelihood and the RSS as a function of beta1. Show visually that ML
```

```
E and OLS are equivalent for linear regression.
beta1_values = np.linspace(-2, 0, 100)
log_likelihood = []
rss = []
for beta1 in beta1_values:
    y_pred = beta0 + x * beta1
    residuals = y - y_pred
    rss.append(np.sum(residuals**2))
    log_likelihood.append(-0.5 * n * np.log(2 * np.pi * sigma**2) - np.sum(residuals**2)
/ (2 * sigma**2))
plt.figure(figsize=(10, 6))
plt.plot(beta1_values, log_likelihood, label='Log Likelihood', color='green')
plt.plot(beta1_values, rss, label='RSS', color='orange')
plt.xlabel('beta1')
plt.ylabel('Value')
plt.title('Log Likelihood and RSS as a function of beta1')
plt.legend()
plt.show()
```



Exercise 2

Task 5

In [32]:

```
# General hyperparameters
TAU_SQ = 1.0        # Prior variance on weights
SIGMA_SQ = 0.1**2   # Observation noise variance

# Kernel-specific hyperparameters
POLY_DEGREE = 9
RBF_LENGTHSCALE = 0.1

# Plotting settings
NUM_SAMPLES = 5
X_GRID = np.linspace(0, 1, 200).reshape(-1, 1)
```

In [33]:

```
from scipy.spatial.distance import cdist

def polynomial_kernel(x1, x2, degree=POLY_DEGREE):
    """Computes the polynomial kernel k(x, x') = (1 + x*x')^degree."""
    # TODO: Implement the polynomial kernel function
    return (1 + x1 @ x2.T) ** degree

def rbf_kernel(x1, x2, lengthscale=RBF_LENGTHSCALE):
    """Computes the RBF (squared-exponential) kernel."""
    # TODO: Implement the RBF kernel function
    # Hint: You can use scipy.spatial.distance.cdist(x1, x2, 'sqeuclidean') to efficientl
y compute the squared distances
    square_dist = cdist(x1, x2, 'sqeuclidean')
    return np.exp(-square_dist / (2 * lengthscale**2))
```

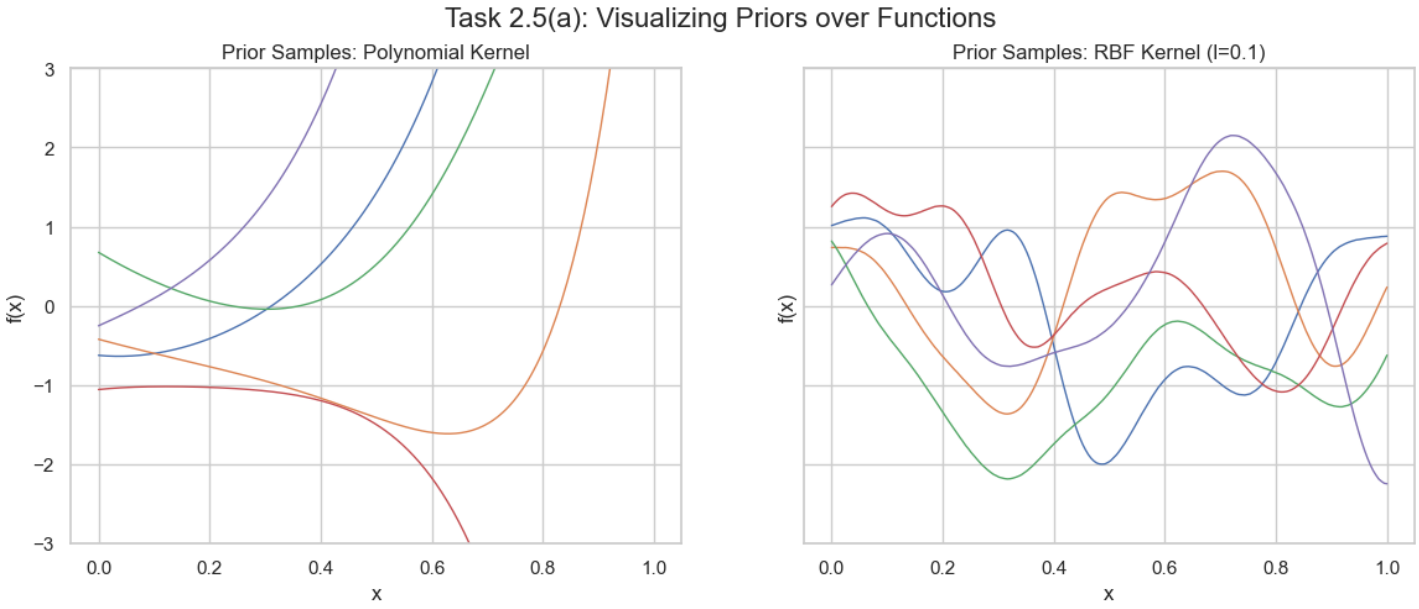
In [34]:

```
from scipy.special import comb

def poly_feature_map(x, degree=POLY_DEGREE):
    """Computes the feature map phi(x) for the polynomial kernel."""
    # TODO: Implement the feature map for the polynomial kernel
    # The d-th feature is sqrt(C(degree, d)) * x^d
    # Hint: A loop over the degree d from 0 to 'degree' is a good approach.
    n = x.shape[0]
    phi = []

    for d in range(degree + 1):
        coef = np.sqrt(comb(degree, d))
        feature = coef * x**d   # shape: (n, 1)
        phi.append(feature)

    return np.hstack(phi)

def sample_from_prior(kernel_func, **kwargs):
    """Samples functions from a GP prior defined by a kernel."""
    if kernel_func == polynomial_kernel:
        # TODO: Implement prior sampling for the Polynomial kernel (weight-space view)
        phi = poly_feature_map(X_GRID)
        w_sample = np.random.multivariate_normal(mean=np.zeros(phi.shape[1]), cov=TAU_SQ
* np.eye(phi.shape[1]), size=NUM_SAMPLES)
        return phi @ w_sample.T
    else:
        # TODO: Implement prior sampling for the RBF kernel (function-space view)
        # Add a small 'jitter' (e.g., 1e-6 * identity matrix) to K for numerical stabilit
y
        K = kernel_func(X_GRID, X_GRID, **kwargs) + 1e-6 * np.eye(len(X_GRID))
        return np.random.multivariate_normal(mean=np.zeros(len(X_GRID)), cov=TAU_SQ * K,
size=NUM_SAMPLES).T
```

In [35]:

```
prior_poly = sample_from_prior(polynomial_kernel)

# Setup the 1x2 plot grid
fig, axes = plt.subplots(1, 2, figsize=(14, 5), sharey=True)
fig.suptitle("Task 2.5(a): Visualizing Priors over Functions", fontsize=16)

# Polynomial Kernel
axes[0].set_title("Prior Samples: Polynomial Kernel")
axes[0].set_xlabel("x")
axes[0].set_ylabel("f(x)")
axes[0].set_ylim(-3, 3) # Set common y-limit for easier comparison

# TODO: Call your `sample_from_prior` function for the polynomial kernel
# and plot the resulting function samples on axes[0].
axes[0].plot(X_GRID, sample_from_prior(polynomial_kernel), lw=1)

# RBF Kernel
axes[1].set_title(f"Prior Samples: RBF Kernel (l={RBF_LENGTHSCALE})")
axes[1].set_xlabel("x")
```

```
axes[1].set_ylabel("f(x)")
axes[1].set_ylim(-3, 3)

# TODO: Call your `sample_from_prior` function for the RBF kernel
# and plot the resulting function samples on axes[1].
axes[1].plot(X_GRID, sample_from_prior(rbf_kernel, lengthscale=RBF_LENGTHSCALE), lw=1)

plt.show()
```



TODO: Briefly comment on qualitative differences between the different kernels. The polynomial kernel produces globally smooth curves, while the RBF kernel produces more locally varying functions.

In [36]:

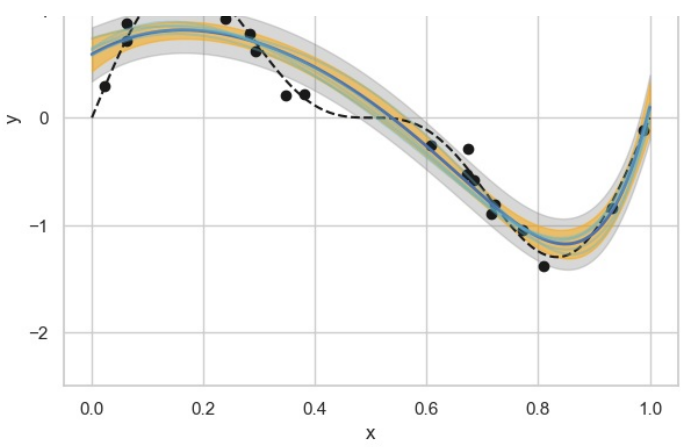
```
def compute_posterior_predictive(X_train, y_train, kernel_func):
    """Computes the mean and variance of the posterior predictive distribution."""
    # TODO: Compute the required kernel matrices:
    # K = k(X_train, X_train)
    # K_star = k(X_GRID, X_train)
    # K_star_star = k(X_GRID, X_GRID)
    K = kernel_func(X_train, X_train)
    K_star = kernel_func(X_GRID, X_train)
    K_star_star = kernel_func(X_GRID, X_GRID)

    K_inv = np.linalg.inv(K + SIGMA_SQ * np.eye(len(X_train)))
    # TODO: Compute the predictive mean using the kernel regression formula on the sheet
    predictive_mean = K_star @ K_inv @ y_train

    # TODO: Compute the epistemic covariance matrix using the formula from the sheet
    epistemic_cov = K_star_star - K_star @ K_inv @ K_star.T

    # Get the point-wise epistemic variance from the diagonal of the covariance matrix
    epistemic_var = np.diag(epistemic_cov)
    # --> Total predictive variance is the sum of epistemic and aleatoric (noise) varianc
e
    total_var = epistemic_var + SIGMA_SQ

    return predictive_mean, total_var, epistemic_var, epistemic_cov

def sample_from_posterior(mean, cov):
    """Samples functions from the posterior predictive distribution."""
    # TODO: Draw NUM_SAMPLES from the multivariate normal distribution
    # defined by the predictive mean and the epistemic covariance matrix
    # Hint: Add a small jitter to 'cov' before sampling to ensure it is positive definite
.

    jitter = 1e-6 * np.eye(len(mean))
    return np.random.multivariate_normal(mean, cov + jitter, size=NUM_SAMPLES).T
```

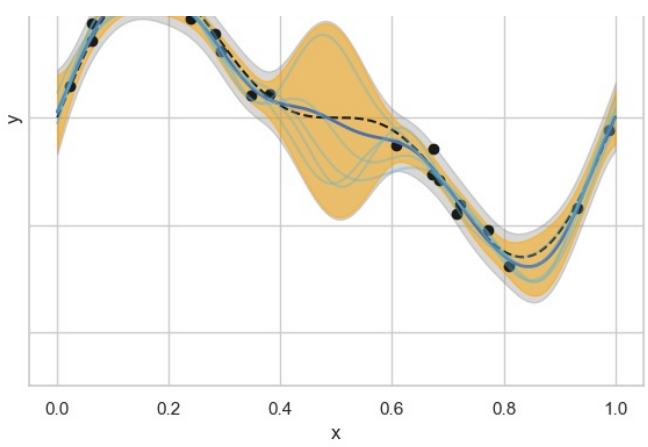
In [37]:

```
def true_function(x):
    return np.sin(2 * np.pi * x) + 0.5 * np.sin(4 * np.pi * x)

def generate_data_with_gap(n=20, noise_std=np.sqrt(SIGMA_SQ)):
    """Generates data with a gap in the middle."""
    np.random.seed(42)
    x1 = np.random.uniform(0.0, 0.4, n // 2)
    x2 = np.random.uniform(0.6, 1.0, n // 2)
    X_train = np.concatenate([x1, x2]).reshape(-1, 1)
    y_train = true_function(X_train.flatten()) + np.random.normal(0, noise_std, n)
    return X_train, y_train
```

In [39]:

```
X_train, y_train = generate_data_with_gap()

fig, axes = plt.subplots(1, 2, figsize=(15, 6), sharey=True)
fig.suptitle(" Posterior Distributions", fontsize=16)

# A dict to cleanly loop over the two kernel models
kernels_to_test = {
    "Polynomial": (polynomial_kernel, {'degree': POLY_DEGREE}),
    "RBF": (rbf_kernel, {'lengthscale': RBF_LENGTHSCALE})
}

# 3. Loop through each kernel, compute its posterior, and plot
for ax, (name, (kernel_func, kwargs)) in zip(axes, kernels_to_test.items()):

    # TODO: Call your functions to get the posterior predictive distribution
    # and to draw samples from the posterior.

    mean, total_var, epistemic_var, epistemic_cov = compute_posterior_predictive(X_train,
y_train, kernel_func=kernel_func)
    posterior_samples = sample_from_posterior(mean, epistemic_cov)

    # Calculate standard deviations for plotting 95% Gaussian credible intervals
    total_std = np.sqrt(total_var)
    epistemic_std = np.sqrt(epistemic_var)

    # Plot true function and training data
    ax.plot(X_GRID, true_function(X_GRID.flatten()), 'k--', label="True Function")
    ax.plot(X_train, y_train, 'ko', label="Training Data")

    # Plot predictive mean and credible intervals
    ax.plot(X_GRID, mean, 'b-', lw=2, label="Predictive Mean")
    ax.fill_between(X_GRID.flatten(), mean - 1.96 * total_std, mean + 1.96 * total_std,
                    color='gray', alpha=0.3, label="Total Uncertainty")
    ax.fill_between(X_GRID.flatten(), mean - 1.96 * epistemic_std, mean + 1.96 * epistem
ic_std,
                    color='orange', alpha=0.5, label="Epistemic Uncertainty")

    # Plot posterior function samples
    ax.plot(X_GRID, posterior_samples, 'c-', alpha=0.4)

    # Final plot settings
    ax.set_title(f"Posterior: {name} Kernel")
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.legend(loc='upper left')
    ax.set_ylim(-2.5, 2.5)

plt.show()
```





TODO: Discuss the results.

- **Which kernel provides a more reasonable fit to the data and why? The RBF provides a more reasonable fit because it is a lot more close to the pattern in the data**
- **Compare the epistemic uncertainty for both models. Where is it largest? How does it behave inside the data gap you created? Epistemic uncertainty is larger between x = 0.4 and 0.6, it looks like there's no training points eixst. In the RBF model, it behaves more variant during the data gap.**
- **How do the posterior function samples relate to the uncertainty bands? Explain what the spread of these samples represents. The posterior samples mostly stay within the uncertainty bands. Their spread is wider where the model is less confident**